Exercise – 1 Singleton Pattern:

Main.java:

```
public class Main {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("First log message");
        logger2.log("Second log message");

        // Prove they're the same instance
        System.out.println("Same instance? " + (logger1 == logger2));
        System.out.println("logger1 hashcode: " + logger1.hashCode());
        System.out.println("logger2 hashcode: " + logger2.hashCode());
    }
}
```

Logger.java:

```
import java.time.LocalDateTime;

public class Logger {

    // Single instance, volatile so changes are visible across threads
    private static volatile Logger instance;

    // Private constructor to prevent instantiation from outside
```

```
    private Logger() {
        System.out.println("Logger instance created.");
    }

    // Thread-safe accessor
    public static Logger getInstance() {
        if (instance == null) {
            synchronized (Logger.class) {
                if (instance == null) {
                    instance = new Logger();
                }
            }
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[" + LocalDateTime.now() + "] " + message);
    }
}
```

Output: